

SpeakMate - Technical Questions & Answers

1. How does the real-time voice pipeline work, and what are the mechanisms in place to handle high concurrency and reduce latency?

Answer:

The real-time voice pipeline relies on WebRTC via LiveKit Cloud. The process begins on the client-side `/setup/confirm` page, where an optimistic LiveKit token is pre-generated via the Node.js backend. This pre-warms a long-running Python worker process (`LiveKitAgentWorker`) deployed on Modal serverless GPUs.

Once the user joins the room in the frontend (using the `@livekit/components-react` library), their audio stream is established. The Modal Python worker utilizes a "forkserver" execution model to quickly load heavy ML models (`Faster-Whisper/PhoWhisper` for STT and `NeuTTS` for TTS) from cached NFS volumes into VRAM. As the user speaks, Client-Side or Server-Side VAD (Voice Activity Detection) detects speech chunks.

To reduce latency, the system implements:

1. **Model Pre-loading:** Pre-warming models before the user explicitly presses "Start".
2. **Audio Stream Directness:** Bypassing HTTP roundtrips; audio frames negotiate directly between the client browser and the LiveKit Selective Forwarding Unit (SFU) closest to Modal's datacenter.
3. **Data Channels:** Using direct WebRTC data channels to send transcripts and state (e.g., `agent_ready`, `turn_confirmed`) instantly, facilitating optimistic UI updates globally.

If the WebSocket/WebRTC connection fails (e.g., restricted corporate networks), the feature flag `NEXT_PUBLIC_USE_LIVEKIT=false` activates the standard HTTP fallback: capturing audio locally via `MediaRecorder`, sending `audio/webm` via POST to the backend, and utilizing the Modal HTTP server endpoints for sequential STT -> LLM -> TTS processing.

2. Explain the "Architecture v2" improvements regarding Node.js and Python responsibilities. Why were these changes necessary?

Answer:

Before "Architecture v2", the Python pipeline deployed on Modal acted heavily as both the Data Plane (audio parsing) and the Control Plane (state management, direct database connections). This created scaling issues, synchronization drift with the Express backend, and security risks by holding database credentials inside ephemeral serverless workloads.

Architecture v2 addressed this through strict plane separation:

- **Node.js (Control Plane):** Now acts as the Single Source of Truth (SSoT). It retains all database credentials, handles JWT Auth, maintains sliding window contexts (e.g., keeping only the last 6 turns to reduce LLM token usage context windows), enforces rate-limits, and logs conversation turns.
- **Python (Data Plane):** Stripped down to handle only intensive media processing inference. The worker no longer holds DB keys. Instead, it utilizes an `INTERNAL_API_KEY` to securely POST/GET conversation turn contexts and transcripts back to internal endpoints on the Node.js Express server.

This change was necessary to ensure deterministic state tracking, isolate DB connections, safely enable features like "Audio Barge-in" (where accurate char-based context truncation requires perfect synchronization), and ensure strict single responsibilities.

3. How is the Multi-Agent LLM system structured, and how does it maintain conversational coherence without hallucination?

Answer:

SpeakMate uses a domain-driven multi-agent LLM design, primarily leveraging Gemini 2.0/2.5 Flash via the `@google/genai` SDK. The agents are tightly scoped Node.js classes instantiated only inside the `PracticeController`.

1. **BrainAgent:** Used primarily ahead-of-time (AOT) to bootstrap the state. It drafts the `FullScenarioContext` output as strict JSON.
2. **VoiceAgent:** The runtime agent driving the dialogue.
3. **AnalystAgent:** The post-runtime grading agent.

To prevent hallucinations and drift:

- **System Prompt Constraints:** Prompts strictly forbid the LLM from outputting descriptive UI actions (like writing "*smiles*"), restrict identifying as an AI ("As an AI language model..."), and forcefully constrain its identity strictly back to the `Scenario` context.
- **Data Formatting:** All requests and responses map directly to Zod schemas (Contracts like `EvaluationReport` or `DialogueTurn`).
- **Sliding Window Context:** Retaining only the immediate relevant conversational turns (history arrays), offloading massive context histories into background summaries handled by the `bridge_agent.py`.
- **Audio Parameter Tuning:** On the STT side (`Faster-Whisper`), adjusting `compression_ratio_threshold=2.4`, `temperature=0.0`, and setting `condition_on_previous_text=False` aggressively minimizes speech recognition echoing/hallucinations (e.g., common Vietnamese dataset artifacts like "Cảm ơn các bạn đã theo dõi").

4. How does the application implement offline persistence and cross-route state management on the Next.js frontend without Redux/MobX?

Answer:

SpeakMate embraces native React primitives and browser storage rather than relying on heavy global state libraries.

- **React Context API:** The core flow centers around `ScenarioContext` and `LanguageContext`. `ScenarioContext` holds the real-time, optimistic state of the active scenario, history, and generated audio blob markers. This context is strictly scoped to the `/practice` layout constraints.
- **Session Storage / Local Storage:** Transient setup phases (e.g., user boarding profiles or the Mentor Ni interactive chat sessions) use custom React Hooks (like `useStoryChat` and `useMentorChat`) synced explicitly with the Browser's `localStorage` API. This allows users to unexpectedly refresh during onboarding or story generation and resume perfectly without unnecessary database INSERTs.
- **URL Query Params (useSearchParams):** For deeply nested evaluation drill-downs, the application relies on routing state. Passing identifiers through URL boundaries ensures deep linking functions flawlessly and reduces complex cross-component payload transfers.
- **Data Fetching:** Moving between "Setup" -> "Confirmation" -> "Conversation" relies gracefully on React Server Components (where applicable) alongside client-side `ApiClient` fetches caching via normal browser semantics.

5. Describe how the "Story Bank" and "Mentor Ni" gamification integration resolves content tracking in the database?

Answer:

The Story Bank feature assists users in framing their real-world professional experiences into structured formats (STAR, CAR, or PREP).

When a user engages **Mentor Ni**, the `MentorAgent` streams contextually-aware dialogue to guide the user's input. The conversation flow is stored in two DB tables: `mentor_chat_sessions` and `mentor_chat_messages`.

Once the user finishes drafting, the `StoryBankAgent` interprets the entire unstructured chat string, mapping explicit statements to JSON fields representing structural parts (Situation, Task, Action, Result).

This artifact is stored within the `story_entries` table.

Gamification Loop Integration:

If the `BrainAgent` detects specific weaknesses in user proficiency (e.g., "Weak active verb usage"), it generates a `realworld_challenges` record. When the user interacts with the app, the interface searches the `story_entries` table for tags correlating to that challenge. If a match is found, the UI proactively surfaces a modal suggesting they apply their custom "Story Bank" context to the current speaking challenge, weaving learning pathways continuously into practical applications.